

Getting Started with Stata

Workshop 1 · 75 minutes · No statistics background assumed. No prior Stata assumed.

Setting up

1. **Get the files.** Go to tinyurl.com/ndpop-stata-1 → the green **Code** button → **Download ZIP**. Then **unzip** it somewhere you can find again.

2. **Stata is already installed on the lab workstations** — nothing to set up. If you would rather use your own laptop that is fine, as long as Stata is already on it; everything today works the same on Windows and Mac. If you need help installing it, catch me afterwards.

3. **Double-click `first_looks.dta`** inside the folder you just unzipped. **That is how you open Stata today** — opening the data opens the program, with the data already in it. No path to type and nothing to configure. What that quietly did for you is the subject of 2.9.

Lab workstations are shared machines. Before you leave, copy your work to a USB stick, your email, or a cloud drive. That matters more than usual today: Block 4 is yours to take home.

How this sheet works

Type the monospace lines exactly as printed. Nothing on this sheet has a blank to fill in.

Boxes with a bar down the side are marked *Advanced* — *not for today*. They are real, working code, printed so you have it, with no task attached. Run them after the session; every one of them is also in `block2_complete.do` or `block3_complete.do`.

The three blocks hand you less and less, on purpose. Block 2 prints the commands. Block 3 tells you what to do and names the command in brackets. Block 4 tells you what to do and names the *help file*. By Block 4 you are doing what you will actually be doing afterwards: looking something up.

Bold questions have no printed answer. Think, then compare with whoever is next to you. The worked do-files answer them.

Check: lines say what you should see. That is how you check yourself without asking.

Blocks 2 and 3 both use `first_looks.dta`, and nothing you do in Block 2 needs saving. Block 4 imports a CSV instead.

The workflow

what an analysis is made of

COLLECT → INSPECT ↔ CLEAN → ANALYZE → REPORT

COLLECT	not today	— the data was extracted for you
INSPECT	Block 2	in this room
CLEAN	Block 3	in this room
ANALYZE	Block 4	in the file you take home
REPORT	Block 4	in the file you take home

INSPECT and CLEAN are a loop, not two steps in a row. Block 3 is where you run that loop for real.

Four of the five are here today — INSPECT and CLEAN in this room, ANALYZE and REPORT in the Block 4 file you take home.

COLLECT is the one nobody does today. The Google Trends export and the Census extract were prepared before you arrived.

And correlate and regress are two lines each. Knowing *which* two lines, and what they entitle you to say, is the rest of a course and degree. Today is the part that comes before that question — and it is the same regardless of what the question turns out to be. Which is why it is worth learning first, and why Workshop 2 is also entirely on this side of the line.

Block 2 First look at Stata inspect

Work in the Command window. Nothing here is saved, and nothing here can break the data.

2.1 Open it

The data is already open — double-clicking it started Stata. Your first command is just:

```
browse
```

`browse` opens a spreadsheet view. It is read-only — look, do not edit. Close that window to get back to the Command window.

What *is* this data? What does one row represent? What does one column? Answer from the screen alone, before anyone tells you.

Hold onto the row question. It comes back in Block 4, and Workshop 2 is built on it.

2.2 Two small experiments

```
Browse
bro

command Browse is unrecognized
r(199);
```

What are the two lessons?

You have also now seen what an error looks like. Errors are normal. *Read them.*

2.3 The windows

Nothing to type. **Results** is what Stata said. **Command** is where you type. **History** is everything you have run — Block 2 ends by turning it into a file. **Variables** lists what is in the data; double-clicking a name types it for you. **Properties** shows what Stata knows about the variable you have selected.

Use a drop-down menu with the **Results** window in view. Stata writes out the command it just ran. That is how you find a command name you were about to go searching for.

2.4 Describing, and the shape of a command

Run these one at a time and watch what changes.

```
describe
describe state
describe state pop
describe state pop-popurban
describe state pop*
describe state pop??p
```

What is the general shape of a Stata command?

pop-popurban is a *range* through the dataset's own order, not an alphabetical one. pop* is a wildcard; pop??p matches one character per ?.

Advanced — not for today. Ranges only run forwards. pop comes after state in the dataset, so asking for pop-state asks Stata to run backwards. The error is worth seeing once — it is r(111), which you will meet again as *variable not found*.

```
describe state pop-state
```

```
variables out of order
r(111);
```

Three ways of looking at the same variables:

```
codebook state pop has_nd
inspect state pop has_nd
summarize state pop has_nd
```

Check: summarize gives state zero observations. It is text, and there is no mean of a word.

When would you reach for one of those three rather than another?

2.5 Listing rows, and if

```
list state has_nd
list state has_nd if has_nd == 1
```

Check: Indiana, and only Indiana. == asks a question; = assigns a value. They are not interchangeable.

Predict first. Write down what you think each of the next two returns — then run them.

```
list state has_nd if has_nd != 0
list state has_nd if has_nd > 0
```

Were you right? If not, what is going on?

Whatever you conclude, the habit that protects you is the same: run `count if missing(x)` *before* you filter on `x`. `block2_complete.do` explains what happened here — read it once you have argued about it, not before.

```
help list
```

2.6 Counting

```
tabulate border
tabulate state border
tabulate border coastal
tabulate border coastal, row column
```

Check: 35 and 15 on the first one. The second fills the screen with 50 rows, which is itself the lesson about when a cross-tabulation stops being useful.

What is the comma doing?

Add it to your sketch from 2.4.

```
count if border == 1 & coastal == 1
list state if border == 1 & coastal == 1
```

Check: 7 states — Alaska, California, Maine, New Hampshire, New York, Texas, Washington.

2.7 Sorting

Advanced — not for today. Row order. `sort` goes up only; `gsort` takes a minus sign to go down. `in` filters by row *position* and `if` by *value*, so `in` depends on how the data happens to be sorted and `if` never does. The last line gives the five states with the most people over 65 — which is not the same thing as the five oldest states.

```
sort pop
gsort pop65p
gsort -pop65p
list state pop65p in 1/5
```

2.8 Your first do-file

This is the point of the whole block, and it is done by clicking.

1. **History** window → the circular warning icon, which hides the commands that errored → **Copy All**
2. `doedit` — an empty do-file editor opens
3. Paste
4. Save it as `first_do.do`
5. `do first_do.do`

This can turn hours of interactive typing into something you can run again in seconds.

If someone handed you that file tomorrow, could they follow it? What is missing from it?

2.9 Where has Stata been working all this time?

```
cls
pwd
```

Read what it says. You never told Stata where to work — so why does it have an answer?

Because double-clicking a file chose it for you. Now watch what happens when nothing chooses. **Close Stata completely**, and start it again from the **Windows Start menu** (or **Spotlight** on a Mac) — not by double-clicking anything.

```
pwd
```

Check: somewhere else entirely — whatever folder Stata happens to start in. Your data is not there, and nothing you type will find it until you say where to look.

So say where. Open your unzipped folder in File Explorer or Finder, click the address bar, and copy the path. Then type `cd "`, paste the path, and close the quote — it will look like `cd "C:/Users/nd_netid/Downloads/stata-onboarding"`. This is the one line on this sheet that is not typed exactly as printed, because only you know where your folder is. Then check it:

```
pwd
```

Check: `pwd` now prints your workshop folder. **Use forward slashes** — they work on Windows and Mac both, and Windows hands you backslashes when you paste, so change them.

Block 3 opens with that `cd` line, and `do_not_start_here.do` has it written out for when you need it in a hurry.

Block 3 Your first Stata project clean

Open `block3_student.do`. From here you type into the do-file, not the Command window, and you write the syntax yourself — each step names the command in brackets. Run the highlighted line or lines with **Ctrl-D** (Windows) or **Cmd-Shift-D** (Mac).

3.1 Where Stata is working

Run `pwd`. If it is not your workshop folder, `cd` to it as you did in 2.9 — that is what the `cd` line at the top of `block3_student.do` is for. Everything after it is a bare filename.

Advanced — not for today. Two shorthands, and the argument about paths. `.` is where you are; `..` is one level up. A path like `first_looks.dta` breaks the moment you move the file; a path like `C:/Users/you/...` breaks the moment anyone else opens your do-file. Neither is safe on its own, which is why the habit is to `cd` once at the top and use relative names after.

```
cd .
cd ..
pwd
```

3.2 What a do-file starts with

Lines beginning with `*` are comments. Stata ignores them; people read them. Put your own name and today's date in the header of `block3_student.do`. That is the whole exercise, and it is the thing everybody skips.

3.3 Does the data add up?

Population is split by age — `pop1t5`, `pop5_17`, `pop18p`, `pop65p`. Those four should total `pop`.

Add the four age groups together and call the result `check_pop`. (**generate**) Then compare it against `pop`, using the summarizing command from 2.4. (**summarize**)

Check: the means differ — 4,518,149 against 5,027,652.

Your eyes can catch that. They cannot catch it in a dataset with 200 variables. So make Stata check instead:

```
assert pop == check_pop
50 contradictions in 50 observations
assertion is false
r(9);
```

This fails on purpose, and it stops the do-file. That halt is the entire point of `assert`: a summary table you skim past is easy to ignore, and a file that refuses to continue is not.

Why did it fail? Look at what each of the four age variables actually covers.

Once you have worked it out, redefine `check_pop` using only the groups that do not overlap — it already exists, so this is not **generate**. (**replace**) Then `assert` again. `block3_complete.do` says which groups, and why, after you have had a go.

Check: silence. `assert` prints nothing at all when it passes.

Now build the same check from the other side: `pop18p` minus `pop65p`, called `pop18_65`. (**generate**)

Is that name accurate?

(**rename**)

Then build `check2_pop` from the four groups that no longer overlap, `assert` it against `pop`, and remove both check variables. (**generate**, **assert**, **drop**)

3.4 Functions

Advanced — not for today. Stata evaluates maths expressions, and ships a library of functions for numbers, strings and dates. Nobody memorises the list — knowing it exists, and how to search it, is the skill.

```
generate pop_m = pop / 1000000
replace pop_m = round(pop_m, .1)
help functions
```

3.5 Stata remembers more than it shows

Advanced — not for today. Every command leaves its results in memory, and `return list` shows them. They are overwritten by the next command that returns anything, so use them immediately or store them first. `egen` reaches the same answer by a different route — worth knowing, because it usually does.

```
summarize pop, detail
return list
generate big_state = (pop >= r(p90))
list state pop if big_state == 1

egen big_state2 = pctlile(pop), p(90)
replace big_state2 = pop >= big_state2
assert big_state == big_state2
drop big_state2
```

3.6 Text

Advanced — not for today. Advanced string work. Five state names are stored short — N. Carolina, N. Dakota, S. Carolina, S. Dakota, W. Virginia. `regexpm()` asks whether a string matches a pattern, and `"\."` means “contains a full stop”. You are not expected to write regular expressions; recognising one is enough. `inlist()` is the readable way to say “any of these”, and writing it instead as `if state2 == "SC" | "SD"` reads like English but is not valid — “SD” on its own is not a test, and Stata answers `type mismatch, r(109)`. `preserve` and `restore` because that `replace` overwrites `state`, and you want your data back afterwards.

```
preserve
list state state2 if regexpm(state, "\.")
generate state3 = substr(state, 4, strlen(state)) if regexpm(state, "\.")
replace state = "South" if inlist(state2, "SC", "SD")
restore
```

3.7 Labels

Three kinds, broadest to narrowest: the dataset, a variable, and the values inside a variable.

Give the dataset a title (`label data`), and give `coastal` a readable name (`label variable`).

Value labels work differently — you define a named set once, then apply it to as many variables as you like. Run `tabulate coastal border`; then define a set mapping 0 to No and 1 to Yes, apply it to both variables, and run the same `tabulate` again. (`label define, label values`)

Check: same numbers, readable table.

What changed, and what didn't?

Compare the two tables carefully — the more interesting half of the answer is the part that did *not* change.

Advanced — not for today. `region` arrived with a value label already attached, called `cenreg`, and `, modify` edits an existing set in place rather than defining a new one. `nolabel` is singular — `nolabels` gives option `nolabels not allowed, r(198)`. Note that this changes how `region` prints for the rest of your session.

```
tabulate region
label define cenreg 1 "Northeast" 2 "Midwest" 3 "South" 4 "West", modify
tabulate region
tabulate region, nolabel
```

3.8 Groups

`_n` is this row's number. `_N` is the total number of rows. Create `nval` from `_n` and `nvals` from `_N`, look at the first few rows, then drop them both. (`generate, list, drop`)

What are those two variables?

Now put `bysort region:` in front of a command and Stata runs it once per region — which changes what `_n` and `_N` refer to. With `bysort region:`, create `region_states` from `_N`, `nval` from `_n`, and `region_pop` as the total of `pop` in each region. (`generate, generate, egen with sum()`) Then list one row per region instead of all 50. (`list ... if nval == 1`)

Check: four rows — 9, 12, 16 and 13 states. The totals print in scientific notation, because they are raw counts of people. `format region_pop %15.0fc` makes them readable.

Inside `bysort region:`, what does `_N` mean now? And what is `if nval == 1` doing?

This is the idea Workshop 2's collapse is built on.

3.9 Log files

The do-file is what you asked. The log is what Stata answered.

Open a log called `test_log.txt`, print two things to it with `display`, and close it. Three protections belong in there, and working out where each goes is the exercise: `capture` before the close, `,` `text` on the opening, and `,` `replace` on the opening. (`log using, display, log close`)

Check: `test_log.txt` appears in your folder and opens as readable text. Without `,` `text` you get a screenful of `{smcl}` markup instead.

What is each of `capture`, `,` `text` and `,` `replace` protecting you from?

Finally, save your cleaned data as `first_looks_clean.dta`. (`save`)

Block 4 First output from raw input yours to take home

We do not work through Block 4 in the room. It closes the session instead: one figure on the screen, and `block4_student.do` is the file that makes it. It is written to be followed on your own, from cold — nothing in it depends on anything done during the session.

It takes a genuinely raw file — a Google Trends export of search interest for Notre Dame, MIT and Clemson, 272 monthly rows from 2004 to 2026 — and carries it all the way to a figure you could put in a paper. Import a CSV, fix the dates, rename and label, declare it a time series, correlate and regress, draw it, export it.

In Block 2, one row was one state. What is one row here?

Every task in that file ends with a `help:` line naming the help file for each command the answer uses — not the command itself. Read the **Syntax** block at the top and the **Examples** at the bottom; the middle is reference you can skip until you need it. That is the actual skill. Nobody memorises Stata; they know what to look up.

`block4_complete.do` has every answer, commented.

Block 5 Where to go next

Three things to look up: macros (`help macro`) · loops (`help foreach`) · user-written commands (`ssc install`)

Getting unstuck, in order: `help commandname` → search keyword → the internet, pasting the exact error text with the `r(###)` included → a person.

People who will help you:

- **ND Pop Stata Helpdesk** — drop in, no appointment. **Mon–Thu 5–7pm, 4020D Jenkins Nanovic Halls.** Bring your laptop. populationanalytics.nd.edu/resources/stata-support-services
- **Navari Family Center for Digital Scholarship** — Hesburgh Library, 2nd floor NE corner. Consultations on data analysis, visualisation, GIS and text mining. cds@nd.edu

Online: the `help` command, which is the full manuals, already installed · stata.com/support · blog.stata.com · stats.oarc.ucla.edu/stata · stata.com/flyers/stata-cheat-sheets — free PDFs, including one for syntax.

AI chatbots are useful for finding a command name or decoding an error. They also invent options that do not exist, fluently and confidently. **Never run a Stata command from a chatbot until you understand it independently.**

A book, for later: Cameron & Trivedi, *Microeconometrics Using Stata*, 2nd ed. (Stata Press, 2022). A graduate text, well beyond today, but its early chapters on syntax and data management cover the ground you started here. Check the library.

Before you log out: copy your do-files off this machine.

Afterwards **Start with** `block4_student.do` — the take-home. Each `_complete.do` holds the answers, and `README.md` lists the rest. All of it stays on github.com/cassandramerritt/ndpop-stata-workshops. Your own do-files do not — save those yourself.